INTELIGÊNCIA ARTIFICIAL APLICADA

Instituto NTA

FASE 1 · ORQUESTRAÇÃO COM ESTADO

Aula 1.1 · primeira da fase

Do laço ao grafo de estado

O mesmo ciclo da Fase 0, agora declarado em vez de escrito. E o estado sai de uma variável local para um objeto que sobrevive à execução.

Série: Engenharia de IA Aplicada · Agentes e RAG em Produção
Projeto: Núcleo de Agentes Operacionais
Duração de estudo: cerca de 1h30 · 25 min de teoria e 55 de código
Pré-requisito: Fase 0 completa (tag **fase-0**)

Prof. Thiago Azeredo Rodrigues · Série pública · 2026

1. Objetivo de aprendizagem

Ao final desta aula você terá migrado o laço da Fase 0 para um grafo de estado no LangGraph, com o mesmo comportamento verificado por teste, e saberá explicar o que o framework resolve que o `while` não resolvia.

2. Por que esta aula importa

Seu agente funciona. Então por que trocar um laço que funciona por um framework?

A resposta não é performance nem elegância. É esta: na Fase 0, o histórico da conversa vivia numa **variável local** dentro de `rodar_agente`. Ninguém de fora conseguia ler aquilo enquanto rodava, pausar no meio, ou retomar depois. Quando a função retornava, o estado evaporava.

Isso tem três consequências que você sente em produção. Se o processo cai no meio de um atendimento, a conversa se perde inteira. Não há como pausar antes de uma ação sensível, mostrar para um humano e esperar aprovação. E não há como retomar do ponto exato depois de resolver alguma coisa.

Persistência, interrupção e retomada. As três dependem da mesma coisa: **o estado precisa ser um objeto explícito, não uma variável de pilha**. É isso que um grafo de estado entrega, e é o motivo real desta fase.

> **O QUE NÃO MUDA, E É O PONTO**
>
> Esta aula não reescreve o motor. `providers`, `registro`, `ferramentas`, `resiliencia` e `observabilidade` continuam **intocados**. O arquivo novo só orquestra o que a Fase 0 já construiu. Se você precisasse reescrever o motor para usar framework, o motor estaria acoplado ao laço, e a Fase 0 teria falhado.

3. Teoria

3.1 Grafo de estado em três conceitos

Conceito	O que é	Na Aula 0.3 era
Estado	O objeto que percorre o grafo e que cada nó lê e atualiza	a variável local `historico`
Nó	Uma função que recebe o estado e devolve o que mudou	um bloco dentro do `for`
Aresta	Para onde ir depois. Fixa ou condicional	o `if` e o fim do laço

Nada aqui é conceitualmente novo. O que muda é que as três coisas passam a ser **declaradas** em vez de escritas, e por isso o framework consegue mexer nelas: salvar o estado, interromper entre nós, retomar.

3.2 O reducer, que é a parte que pega quem está chegando

Este é o conceito que mais confunde, e ele cabe numa frase: **um nó devolve apenas o que mudou, e o reducer decide como juntar isso ao que já existia.**

Sem reducer, um nó que devolve `historico` **substitui** o valor anterior. Com um reducer de soma, ele **concatena**. É literalmente a diferença entre `historico = novas` e `historico.extend(novas)` da Aula 0.3, com a diferença de que agora isso é declarado no tipo do estado, uma vez, em vez de repetido em cada lugar que mexe na lista.

> **ANALOGIA**
> Pense num processo que passa por várias mesas. Cada mesa não devolve a pasta inteira refeita: devolve a folha que acrescentou. O reducer é a regra de arquivamento que diz o que fazer com cada folha que chega: anexar ao final, substituir a anterior, somar ao total. Declarar a regra uma vez evita que cada mesa reinvente o arquivamento.

3.3 Uma decisão: por que não usamos o reducer pronto

O LangGraph oferece um reducer chamado `add_messages`, que todo tutorial usa. Nós **não** vamos usar, e a razão é a mesma que organiza o curso inteiro.

Aquele reducer é feito para objetos de mensagem do LangChain. Usá-lo obrigaria a converter o nosso histórico para aquele formato, e o motor da Fase 0, que hoje é agnóstico de provider, passaria a depender do formato de mensagem de um framework específico.

O nosso histórico é uma lista de dicionários no dialeto de cada provider. Um reducer de soma simples acumula exatamente igual, sem acoplamento nenhum. Vale a pena saber que o `add_messages` existe, porque você vai encontrá-lo em toda documentação; e vale saber por que, aqui, ele cobraria um preço que não queremos pagar.

3.4 A regra do estado: tudo que entra precisa ser serializável

Esta regra não é burocracia: é a razão de o estado existir. Se o estado vai ser salvo por um checkpointer na Aula 1.6, tudo que estiver dentro dele precisa saber virar bytes.

Duas consequências práticas aparecem no código de hoje. As **ferramentas e os executores não moram no estado**: são funções Python, e funções não serializam. Eles são capturados por closure na construção do grafo. E os pedidos de ferramenta são guardados como **dicionários puros**, não como os nossos objetos: um checkpointer reclama de classe própria, e o nó que precisa dos objetos os reconstrói na hora.

> **UMA DÍVIDA ASSUMIDA, E NOMEADA**
> Sobra um campo problemático: o turno cru do assistant, que a API exige de volta na devolução (Aula 0.2) e que guarda objetos do SDK do provider. Com checkpointer em memória funciona; num checkpointer de banco, na Aula 1.6, vai ser preciso serializá-lo. Deixamos o problema **visível e nomeado no código** em vez de escondido. Dívida que ninguém escreveu é dívida que alguém descobre em produção.

3.5 O teto de iterações mudou de nome

Na Aula 0.3 você escreveu `max_iteracoes` e o laço parava. O LangGraph chama isso de `recursion_limit` e levanta uma exceção própria ao estourar.

Mesmo conceito, outro nome, com um detalhe de conversão: no grafo, cada volta do ciclo consome **dois passos** (decidir e agir), mais o passo final. Quem traduz um pelo outro é o nosso código, para quem chama continuar pensando em voltas.

4. Mão na massa

Esta aula cria **um arquivo novo**, `nucleo/grafo.py`, e seus testes. Nenhum arquivo da Fase 0 é alterado.

Antes de começar: `pip install langgraph`. O código abaixo foi verificado contra a versão 1.2.15. A API deste pacote muda entre versões maiores, então confira a sua com `pip show langgraph` e, na dúvida sobre uma assinatura, `inspect.signature` continua sendo a fonte da verdade, como na Aula 0.1.

Passo 1 — Declarar o estado

O que e por quê: a mudança conceitual da aula inteira. O que vivia na pilha de uma função vira um objeto que o grafo carrega.

```
class EstadoAgente(TypedDict):
    """O estado que percorre o grafo."""

    historico: Annotated[list, operator.add]
    # O Annotated com um REDUCER e o coracao do LangGraph. Sem ele, um
    # no que devolve `historico` SUBSTITUIRIA o valor anterior. Com
    # operator.add, cada no devolve apenas o DELTA e o framework
    # concatena. E o `historico.extend(...)` da Aula 0.3, agora
    # declarado em vez de escrito a mao.

    iteracoes: Annotated[int, operator.add]
    # Mesmo mecanismo com numero: cada volta devolve 1, o reducer soma.

    provider: str          # "anthropic" ou "openai"
    system: Optional[str]  # instrucao de sistema, opcional
    resposta: str          # texto final, preenchido na ultima volta

    pedidos: list
    # Os pedidos da volta corrente, como DICTS puros, nao como objetos
    # PedidoFerramenta. Nao e preciosismo: um checkpointer precisa
    # serializar isto, e dataclass custom vira aviso hoje e erro
    # amanha. O no `agir` reconstroi os objetos quando precisa deles.

    turno_assistant: Any
    # O turno cru do assistant, exigido pela API na devolucao (Aula
    # 0.2). DIVIDA ASSUMIDA: este campo guarda objetos do SDK do
    # provider, que NAO sao serializaveis. Com o checkpointer em
    # memoria funciona; num checkpointer de banco, na Aula 1.6, vai
    # ser preciso serializar isto. Deixamos o problema visivel e
    # nomeado em vez de escondido.
```

O que faz: o `Annotated` com um reducer é o coração do LangGraph. Repare que o mesmo mecanismo vale para número: cada volta devolve 1, e o reducer soma. E leia os comentários dos dois últimos campos: são a regra da seção 3.4 virando decisão de código.

```
# Por que nao usamos o `add_messages` que o LangGraph oferece: aquele
# reducer e feito para objetos de mensagem do LangChain e faria o nosso
# motor depender daquele formato. O nosso historico e uma lista de
# dicts no dialeto de cada provider, e e assim que a Fase 0 o mantem
# agnostico. operator.add acumula exatamente igual, sem acoplamento.
```

Passo 2 — O nó que decide

O que e por quê: é a etapa DECISÃO da Aula 0.3, sem uma linha nova de lógica.

```
def decidir(estado: EstadoAgente) -> dict:
    volta = estado["iteracoes"] + 1

    # Chamada ao modelo e generation: e nela que mora o custo.
    with observar(f"decisao {volta}", tipo="generation") as gen:
        resposta = chat_ferramentas(
            estado["provider"],
            estado["historico"],
            ferramentas,
            system=estado.get("system"),
        )
        registrar_uso(gen, resposta.tokens_entrada,
                      resposta.tokens_saida, resposta.modelo)

    # O no devolve apenas o que MUDOU. Quem acumula e o reducer.
    return {
        "iteracoes": 1,
        # Guardamos dicts, nao objetos: ver a nota no EstadoAgente.
        "pedidos": [{"id": p.id, "nome": p.nome,
                     "argumentos": p.argumentos}
                    for p in resposta.pedidos],
        "turno_assistant": resposta.mensagem_assistant,
        "resposta": resposta.texto if not resposta.pedidos else "",
    }

# ---------------------------------------------------------------------
# NO 2 - AGIR. E a etapa ACAO mais a OBSERVACAO da Aula 0.3.
```

O que faz: chama o mesmo `chat_ferramentas` de sempre e registra os tokens no mesmo tracing de sempre. A única diferença real está no `return`: ele devolve o **delta**, não o estado inteiro. Quem acumula é o reducer.

Passo 3 — O nó que age

O que e por quê: é a etapa AÇÃO mais a OBSERVAÇÃO da Aula 0.3.

```
def agir(estado: EstadoAgente) -> dict:
    # Reconstruimos os objetos a partir dos dicts do estado. O
    # motor da Fase 0 continua recebendo exatamente o que sempre
    # recebeu, e o estado continua serializavel.
    pedidos = [PedidoFerramenta(**p) for p in estado["pedidos"]]
    resultados = []

    for pedido in pedidos:
        # Execucao de ferramenta e span: tem duracao, nao tem token.
        with observar(pedido.nome, entrada=pedido.argumentos) as sp:
            funcao = executores.get(pedido.nome)
            if funcao is None:
                # Ferramenta desconhecida nao derruba o grafo: vira
                # observacao que o modelo le e contorna (Aula 0.6).
                saida = {"erro": f"ferramenta desconhecida: "
                                 f"{pedido.nome}"}
            else:
                saida = funcao(**pedido.argumentos)
            if sp is not None:
                sp.saida = saida

        resultados.append(
            (pedido, json.dumps(saida, ensure_ascii=False)))
```

```
        # mensagens_de_resultado e da Aula 0.3, reaproveitada inteira:
        # ela sabe o dialeto de devolucao de cada provider. Ela so
        # precisa do turno cru, entao montamos um RespostaLLM efemero
        # em vez de carregar o objeto inteiro no estado.
        novas = mensagens_de_resultado(
            estado["provider"],
            RespostaLLM("", pedidos, estado["turno_assistant"]),
            resultados)

        return {"historico": novas}   # o reducer concatena ao que havia

    # ---------------------------------------------------------------
    # A ARESTA CONDICIONAL. E o `if not resposta.pedidos` da Aula 0.3,
    # agora como funcao de roteamento: ela nao executa nada, so decide
    # para onde ir.
```

O que faz: executa cada pedido com os executores protegidos da Aula 0.6 e devolve as mensagens no dialeto certo com a função da Aula 0.3. Repare nas duas reconstruções: os objetos de pedido e o objeto de resposta são montados **dentro do nó**, a partir do que o estado guarda em formato serializável. O motor recebe exatamente o que sempre recebeu.

Passo 4 — As arestas e a montagem

O que e por quê: a parte que substitui o `while`. Leia as últimas linhas e compare com o laço da Aula 0.3: é o mesmo ciclo.

```
    def tem_ferramenta_a_executar(estado: EstadoAgente) -> str:
        return "agir" if estado["pedidos"] else END

    # ---------------------------------------------------------------
    # A MONTAGEM. Leia as quatro linhas abaixo e compare com o laco da
    # Aula 0.3: e o mesmo ciclo, declarado em vez de escrito.
    # ---------------------------------------------------------------
    construtor = StateGraph(EstadoAgente)
    construtor.add_node("decidir", decidir)
    construtor.add_node("agir", agir)

    construtor.add_edge(START, "decidir")
    construtor.add_conditional_edges(
        "decidir", tem_ferramenta_a_executar, {"agir": "agir", END: END})
    construtor.add_edge("agir", "decidir")   # e o ciclo que fecha o laco

    return construtor.compile(checkpointer=checkpointer)
```

O que faz: a função de roteamento não executa nada, só decide para onde ir, e é o `if not resposta.pedidos` da Aula 0.3 isolado. A aresta de `agir` de volta para `decidir` é o que fecha o ciclo: é literalmente o laço, desenhado.

E porque o grafo é um objeto, a topologia pode ser impressa:

```
graph TD;
    __start__([__start__]):::first
    decidir(decidir)
    agir(agir)
    __end__([__end__]):::last
    __start__ --> decidir;
    agir --> decidir;
    decidir -.-> __end__;
    decidir -.-> agir;
```

Isso o `while` nunca permitiu. Topologia que se desenha é topologia que se discute em revisão de arquitetura.

Passo 5 — Ver os dois lado a lado

O que e por quê: o momento da aula. O mesmo caso, pelos dois caminhos.

```
# scratch/rascunho_1_1.py  (descartavel: o .gitignore ignora scratch/)
from nucleo import banco, ferramentas            # registra o dominio
from nucleo.registro import schemas
from nucleo.resiliencia import executores_seguros
from nucleo.agente import rodar_agente           # Fase 0: o laco
from nucleo.grafo import rodar_grafo             # Fase 1: o grafo

banco.criar_schema()

SISTEMA = """Voce e o assistente comercial do FlowDesk, um SaaS B2B.
Consulte o CRM antes de agir. Problema tecnico vira ticket."""

PERGUNTA = "A Ana do TechFlow relatou erro 500 no login."

# O MESMO caso, pelos dois caminhos.
pelo_laco = rodar_agente(PERGUNTA, schemas(), executores_seguros(),
                         system=SISTEMA, verbose=False)
pelo_grafo = rodar_grafo(PERGUNTA, schemas(), executores_seguros(),
                         system=SISTEMA)

print("laco :", pelo_laco.iteracoes, "voltas |", pelo_laco.parou_por)
print("grafo:", pelo_grafo.iteracoes, "voltas |", pelo_grafo.parou_por)
print()
print(pelo_grafo.resposta)

# O que o grafo tem e o while nao tinha: a topologia e um objeto.
from nucleo.grafo import construir_grafo
g = construir_grafo(schemas(), executores_seguros())
print()
print(g.get_graph().draw_mermaid())
```

Resultado esperado: as duas linhas imprimem o mesmo número de voltas e o mesmo motivo de parada. O comportamento não mudou, e era para não mudar mesmo: o que mudou foi quem guarda o estado.

Passo 6 — O teste que vale a aula

O que e por quê: entre os oito testes do arquivo, um é o que importa de verdade.

```
# ---------- 2. O TESTE DA AULA: grafo e laco dao o mesmo resultado ----------

def test_grafo_equivale_ao_laco_da_fase_0(monkeypatch):
    """Mesma entrada, mesmas decisoes, mesmo resultado.

    Se este teste falhar, a migracao mudou o comportamento, e aí nao e
    migracao: e reescrita.
    """
    _roteiro(monkeypatch, _duas_voltas(), agente)
    pelo_laco = agente.rodar_agente("O Carlos esta qualificado?",
                                    schemas(), executores_seguros(),
                                    verbose=False)

    _roteiro(monkeypatch, _duas_voltas(), grafo)
    pelo_grafo = grafo.rodar_grafo("O Carlos esta qualificado?",
```

```
                                  schemas(), executores_seguros())

    assert pelo_grafo.resposta == pelo_laco.resposta
    assert pelo_grafo.iteracoes == pelo_laco.iteracoes
    assert pelo_grafo.parou_por == pelo_laco.parou_por
    assert len(pelo_grafo.historico) == len(pelo_laco.historico)
```

Se este teste falhar, a migração mudou o comportamento, e aí não é migração: é reescrita. Em qualquer refatoração para framework, este é o teste que você escreve primeiro.

Passo 7 — O gancho que justifica a fase inteira

O que e por quê: aqui você vê, pela primeira vez, o que o grafo entrega e o laço não entregava.

```
# ---------- 8. o gancho da Aula 1.6: o estado sobrevive ----------

def test_estado_sobrevive_com_checkpointer(monkeypatch):
    from langgraph.checkpoint.memory import InMemorySaver

    memoria = InMemorySaver()
    g = grafo.construir_grafo(schemas(), executores_seguros(),
                              checkpointer=memoria)
    monkeypatch.setattr(grafo, "chat_ferramentas",
                        lambda *a, **k: RespostaLLM("Oi.", [], None))

    config = {"configurable": {"thread_id": "lead-ana"}}
    grafo.rodar_grafo("oi", schemas(), executores_seguros(),
                      grafo=g, config=config)

    # A execucao terminou, e o estado CONTINUA LA, recuperavel pelo id
    # da conversa. Na Fase 0 isso morria com a variavel local. E esta
    # a porta para persistencia e human-in-the-loop.
    estado = g.get_state(config)
    assert estado.values["iteracoes"] == 1
    assert estado.values["historico"][0]["content"] == "oi"
```

O que faz: a execução terminou e o estado **continua lá**, recuperável pelo identificador da conversa. Na Fase 0 isso morria com a variável local. Não é truque de framework: é consequência direta de o estado ser um objeto declarado em vez de uma variável de pilha.

Guarde o `thread_id`: é por ele que a Aula 1.6 vai retomar uma conversa e a 1.7 vai devolver o controle a um humano no meio dela.

Passo 8 — Commit e tag

```
pip install langgraph
pip freeze > requirements.txt   # trave a versao da sua maquina
pytest -v                       # esperado: 66 passed, 2 deselected
git add .
git commit -m "Aula 1.1: do laco ao grafo de estado"
git tag aula-1.1
git push && git push --tags
```

5. Portabilidade

Grafo de estado não é invenção do LangGraph: é máquina de estados com outro nome, e você já viu isso em workflow engine, em orquestrador de pipeline e em qualquer sistema de aprovação. O

vocabulário muda (nó, task, step, activity), o desenho é o mesmo.

O que é portável daqui: separar **o que decide** (roteamento) do **que executa** (nós); devolver delta em vez de estado inteiro, com a regra de junção declarada num lugar só; e a exigência de que estado durável seja serializável, que vale para Temporal, para Airflow e para qualquer coisa que prometa retomar de onde parou.

E a lição que a aula demonstra na prática: framework bom substitui a **orquestração**, não o domínio. Se adotar um exige reescrever o seu motor, ou o framework é invasivo demais, ou o seu motor estava acoplado à orquestração desde o começo.

6. Terminei quando

- O grafo resolve o mesmo caso da Fase 0, com o mesmo número de voltas e a mesma resposta.
- O teste de equivalência entre laço e grafo passa.
- Você imprime a topologia do grafo em Mermaid.
- Com um checkpointer em memória, o estado continua recuperável depois da execução.
- `pytest` mostra 66 passed, 2 deselected.

7. Erros comuns e troubleshooting

- **O histórico some ou fica só com a última mensagem.** Faltou o reducer no `Annotated`. Sem ele, o nó substitui em vez de acumular, e é o erro número um de quem está começando com LangGraph.
- **Aviso de tipo não registrado ao usar checkpointer.** Algo não serializável entrou no estado. É a regra da seção 3.4, e o aviso diz exatamente qual classe.
- **O grafo nunca termina.** A função de roteamento não está devolvendo `END` em nenhum caminho, ou o `path_map` não cobre o valor devolvido.
- **Erro de recursão com um número que não bate com o seu teto.** Lembre que cada volta consome dois passos no grafo. É a conversão da seção 3.5.
- **Assinatura diferente da apostila.** Versão diferente do pacote. Confira com `pip show langgraph` e inspecione a assinatura, como na Aula 0.1.
- **Tentação de mexer no motor para o grafo funcionar.** Se você sentir essa necessidade, pare e olhe de novo: o grafo deve **orquestrar** o motor, não modificá-lo.

8. Ponte para a próxima aula

Você tem um grafo com estado explícito, e viu o estado sobreviver à execução com um checkpointer em memória. Mas memória é memória: reinicie o processo e tudo some.

Na **Aula 1.2** a fase continua pelos padrões de workflow, encadeamento, roteamento e paralelização, antes de chegar na persistência de verdade. O `thread_id` que você usou hoje volta lá, apontando para um banco em vez de um dicionário na memória.